

Java Backend Architecture

Interview Series

Chapter 10.10

Search Alternatives

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

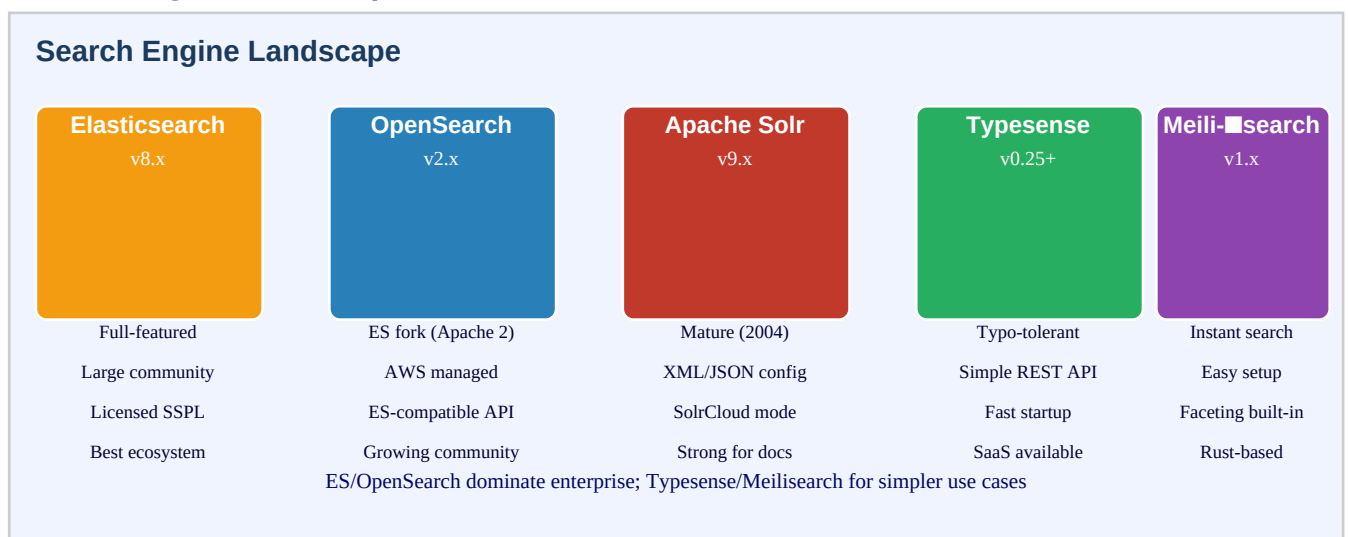
Chapter 10.10 – Search Alternatives

Introduction

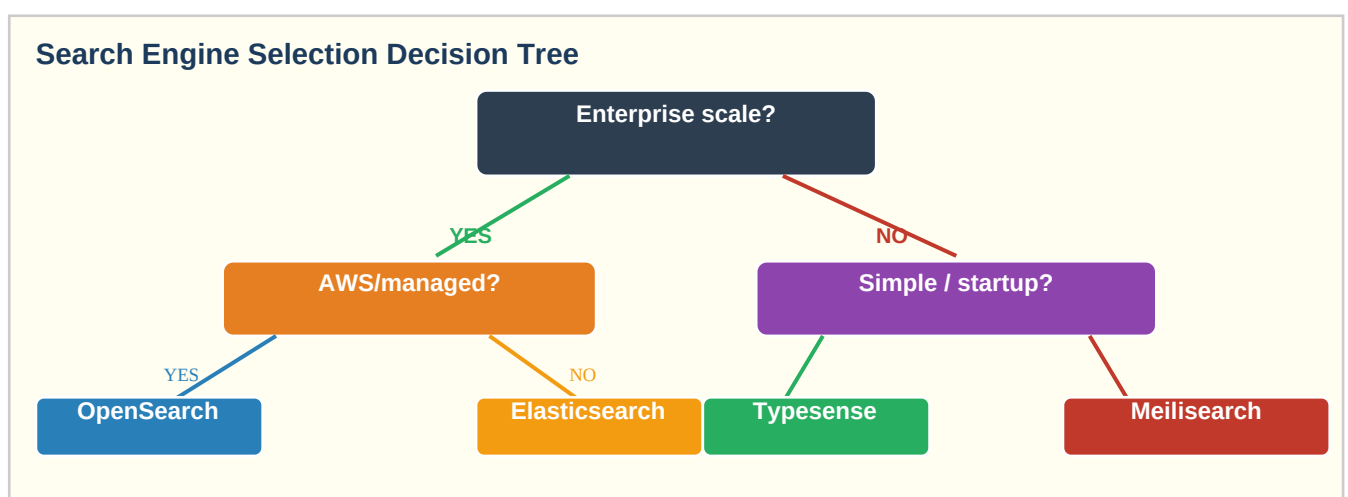
Elasticsearch is the dominant search platform in backend engineering, but it is not the only choice. OpenSearch (its Apache 2.0 fork), Apache Solr (the original Lucene-based search platform), and newer alternatives like Typesense and Meilisearch serve different segments of the market. The rise of AI/ML has also introduced vector search as a first-class capability, with purpose-built vector databases (Pinecone, Weaviate, Qdrant) challenging traditional inverted-index search engines for semantic search workloads.

For a Java backend engineer, understanding the landscape helps in choosing the right tool and in answering architecture trade-off questions in system design interviews. In practice, Elasticsearch (or its fork OpenSearch on AWS) covers the vast majority of enterprise use cases. The newer tools shine for simple SaaS applications that need fast setup with minimal operational complexity.

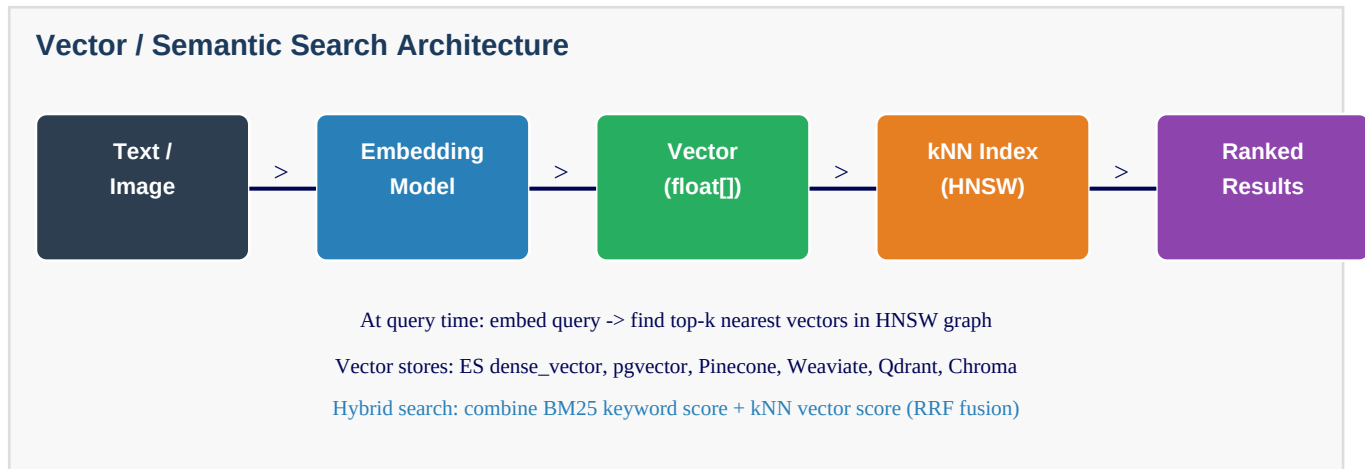
Search Engine Landscape



Selection Decision Tree



Vector / Semantic Search Architecture



Key Definitions

OpenSearch	AWS-backed fork of ES 7.10.2 under Apache 2.0 license; API-compatible with ES 7.x
Apache Solr	Apache Foundation search platform (2004); built on Lucene like ES; XML/JSON config
SolrCloud	Distributed Solr mode using ZooKeeper for coordination
Typesense	Modern, typo-tolerant search engine in C++; simple REST API; SaaS offering
Meilisearch	Instant, typo-tolerant search in Rust; easy config; designed for front-end use
pgvector	PostgreSQL extension adding vector column type and approximate nearest neighbor queries
vector search	Finding documents by semantic similarity using embedding vectors and ANN algorithms
embedding	Fixed-length numeric representation of text/image from an ML model
kNN (k-Nearest)	Finding k most similar vectors to a query vector by distance
HNSW	Hierarchical Navigable Small World — graph-based ANN index algorithm used by ES/pgvector
ANN	Approximate Nearest Neighbor — faster than exact kNN; small accuracy trade-off
hybrid search	Combining BM25 keyword score + vector similarity score for better relevance
RRF	Reciprocal Rank Fusion — algorithm to merge ranked lists from different retrievers
dense_vector	ES field type for storing embedding vectors; supports exact and approx kNN
SSPL	Server Side Public License — ES license since v7.11; not OSI-approved open source

Comparison Table

Engine	License	Language	Strength	Weakness	Managed?
Elasticsearch	SSPL/Elastic	Java	Full-featured, ecosystem	Complexity, license	Elastic Cloud
OpenSearch	Apache 2.0	Java	ES-compatible, AWS-managed	Lags ES features	AWS OpenSearch
Apache Solr	Apache 2.0	Java	Mature, document-centric	Config verbosity	Self-hosted only
Typesense	GPL 3	C++	Typo-tolerant, fast start	Less mature/scale	Typesense Cloud
Meilisearch	MIT	Rust	Instant, ease of use	Limited analytics	Meilisearch Cloud
pgvector	PostgreSQL	C	In-DB vector search	Not full-text	AWS RDS, Supabase

Code Examples

1. OpenSearch — Java client (AWS SDK)

```
// Maven: org.opensearch.client:opensearch-java:2.x // OpenSearch Java client is
// AWS-maintained, API-similar to ES Java client
import org.opensearch.client.opensearch.OpenSearchClient;
import org.opensearch.client.opensearch.core.*;
import org.opensearch.client.opensearch._types.query_dsl.*; // Config for AWS OpenSearch (IAM auth)
SdkHttpClient httpClient = ApacheHttpClient.builder().build();
OpenSearchClient client = new OpenSearchClient(
    new AwsSdk2TransportOptions.Builder()
        .awsCredentials(DefaultCredentialsProvider.create())
        .region(Region.US_EAST_1)
        .serviceName("es")
        .build(),
    new AwsSdk2Transport(httpClient,
        "your-endpoint.us-east-1.es.amazonaws.com", "es", Region.US_EAST_1,
        AwsSdk2TransportOptions.builder().build())
); // Same Query DSL as ES:
SearchResponse<ProductDocument> response = client.search(s -> s.index("products")
    .query(q -> q.match(m -> m.field("name").query("running shoes"))), ProductDocument.class);
```

2. Apache Solr — Java SolrJ client

```
// Maven: org.apache.solr:solr-solrj:9.x
import org.apache.solr.client.solrj.impl.CloudSolrClient;
import org.apache.solr.client.solrj.request.CollectionAdminRequest;
import org.apache.solr.client.solrj.response.QueryResponse;
import org.apache.solr.common.SolrDocument;
import org.apache.solr.common.params.MapSolrParams;

public class SolrSearchService {
    private final CloudSolrClient solrClient;

    public SolrSearchService() {
        // SolrCloud with ZooKeeper
        this.solrClient = new CloudSolrClient.Builder(
            List.of("zk1:2181", "zk2:2181"),
            Optional.of("/solr")
        ).build();
        this.solrClient.setDefaultCollection("products");
    }

    public List<SolrDocument> search(String query, String category)
        throws Exception {
        MapSolrParams params = new MapSolrParams(
            Map.of(
                "q", "name:" + query,
                "fq", "category:" + category, // fq = filter query (cached)
                "fl", "id,name,price,score", // field list
                "rows", "20",
                "sort", "score desc",
                "hl", "true", // highlighting
                "hl.fl", "name,description"
            )
        );
        QueryResponse response = solrClient.query(params);
        return response.getResults();
    }
}
```

3. Typesense — Java client

```
// Maven: org.typesense:typesense-java:0.x (community client)
import org.typesense.api.*;
import org.typesense.model.*;
// Initialize Client
typesenseClient = new Client(new Configuration(
    List.of(new Node("https", "your.typesense.net", "443")),
    Duration.ofSeconds(5),
```

```
"your-api-key" )); // Create collection (schema) CollectionSchema schema = new
CollectionSchema().name("products").fields(List.of( new Field().name("name").type("string"),
new Field().name("price").type("float"), new
Field().name("category").type("string").facet(true), new Field().name("in_stock").type("bool")
)) .defaultSortingField("price"); typesenseClient.collections().create(schema); // Search –
automatic typo tolerance built-in SearchParameters params = new SearchParameters().q("wireless
headphones") // typos handled automatically .queryBy("name") .filterBy("category:=electronics
&& in_stock:=true") .sortBy("_text_match:desc,price:asc") .page(1) .perPage(20); SearchResult
result = typesenseClient.collections("products").documents().search(params);
```

4. Meilisearch — REST API with Java (OkHttp)

```
// Meilisearch has simple REST API – use OkHttp or any HTTP client // Official Java SDK:
com.meilisearch.sdk:meilisearch-java:1.x import com.meilisearch.sdk.*; import
com.meilisearch.sdk.model.*; Client meiliClient = new Client(new
Config("http://localhost:7700", "masterKey")); // Create index
meiliClient.createIndex("products", "id"); // Index documents String jsonDocuments =
"[{\"id\":\"1\",\"name\":\"Running Shoes\",\"price\":89.99}]";
meiliClient.index("products").addDocuments(jsonDocuments); // Search SearchRequest
searchRequest = SearchRequest.builder().q("running shoes").filter(new String[]{"price > 50",
"in_stock = true"}).facets(new String[]{"category"}).hitsPerPage(20).build(); SearchResult
result = meiliClient.index("products").search(searchRequest); // result.getHits() – instant
results with built-in typo tolerance
```

5. Vector search with pgvector (PostgreSQL)

```
-- Setup: CREATE EXTENSION IF NOT EXISTS vector; CREATE TABLE products ( id UUID PRIMARY KEY,
name TEXT, price NUMERIC, category VARCHAR(100), embedding vector(1536) -- OpenAI ada-002
dimensions ); CREATE INDEX ON products USING ivfflat (embedding vector_cosine_ops) WITH (lists
= 100); -- Or HNSW (PG 16+): CREATE INDEX ON products USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64); -- Query: find most similar products to a query embedding
SELECT id, name, price, 1 - (embedding <=> '[0.1, 0.2, ...]':vector) AS similarity FROM
products WHERE category = 'electronics' ORDER BY embedding <=> '[0.1, 0.2, ...]':vector LIMIT
10; -- Java: Spring Data with pgvector // Dependency: com.pgvector:pgvector:0.1.x +
spring-data-jpa @Column(columnDefinition = "vector(1536)") private float[] embedding; //
Repository: @Query(value = "SELECT * FROM products ORDER BY embedding <=> CAST(:v AS vector)
LIMIT :k", nativeQuery = true) List<Product> findNearestNeighbors(@Param("v") String
vectorStr, @Param("k") int k);
```

6. Hybrid search in Elasticsearch (BM25 + kNN)

```
// ES 8.x hybrid search: combine keyword + vector search { "query": { "bool": { "must": [ {
"match": { "description": "wireless headphones" } } ], "filter": [ { "term": { "category":
"electronics" } } ] } }, "knn": { "field": "embedding", "query_vector": [0.1, 0.2, 0.3], //
from your embedding model "k": 10, "num_candidates": 100 }, "rank": { "rrf": { "window_size":
50, "rank_constant": 20 } } } // RRF (Reciprocal Rank Fusion) merges the BM25 rank and kNN
rank: // score = 1/(rank_constant + rank_BM25) + 1/(rank_constant + rank_kNN) // Java:
KnnSearchQuery knnQuery = KnnSearchQuery.of(k -> k .field("embedding")
.queryVector(queryEmbedding) .k(10) .numCandidates(100) ); SearchResponse<ProductDocument>
resp = esClient.search(s -> s .index("products") .query(q -> q.bool(b -> b .must(m ->
m.match(mt -> mt.field("description").query(userQuery))) .filter(f -> f.term(t ->
t.field("category").value("electronics")))) .knn(knnQuery) .rank(r -> r.rrf(rrf ->
rrf.windowSize(50).rankConstant(20L))), ProductDocument.class );
```

Interview Questions & Answers

Q1. What is OpenSearch and how does it differ from Elasticsearch?

OpenSearch is a community-driven fork of Elasticsearch 7.10.2, created by AWS in 2021 when Elastic changed ES/Kibana licensing from Apache 2.0 to SSPL (Server Side Public License). OpenSearch is licensed under Apache 2.0. Differences: (1) API is mostly compatible with ES 7.x — most ES 7.x clients work with OpenSearch. (2) OpenSearch has diverged for new features (ML inference, new dashboards). (3) AWS OpenSearch Service is the managed offering. Choose OpenSearch if you're on AWS or need fully open-source licensing.

Q2. Why did Elastic change to SSPL licensing and what does it mean?

Elastic changed ES and Kibana licenses from Apache 2.0 to SSPL in January 2021, citing cloud providers (specifically AWS) offering hosted ES services without contributing back. SSPL requires that if you provide the software as a service, you must open-source your entire service stack — effectively preventing non-Elastic managed service providers. SSPL is not approved by OSI as open source. For most enterprise users who self-host or use Elastic Cloud, the license change has no practical impact.

Q3. When would you choose Apache Solr over Elasticsearch?

Solr is worth considering when: (1) you have an existing Solr deployment — migration risk isn't justified. (2) You need mature document-processing features (rich parsing, OCR via Tika). (3) You require specific Solr features like SolrCloud's native ZooKeeper integration. (4) Your team has deep Solr expertise. In most new projects, Elasticsearch has a better ecosystem, more active development, and better cloud/managed options. Solr has been losing market share to ES but remains strong in content management and enterprise search.

Q4. What are the key differences between Solr and Elasticsearch architecturally?

Both are built on Apache Lucene. Differences: (1) Config: Solr uses XML schema.xml + solrconfig.xml; ES uses JSON API/settings. (2) Coordination: Solr uses ZooKeeper for cluster state; ES has its own cluster coordination. (3) REST API: ES has a unified, intuitive JSON REST API; Solr's API is more complex. (4) Query DSL: ES has a rich composable DSL; Solr uses Lucene query string syntax + edismax. (5) Aggregations: ES has more powerful aggregation framework than Solr's faceting.

Q5. What is Typesense and when would you choose it?

Typesense is a modern, typo-tolerant search engine written in C++. Key features: (1) Built-in typo tolerance without configuration. (2) Very simple setup (single binary). (3) Fast startup — suitable for development and testing. (4) Clean REST API. (5) SaaS offering (Typesense Cloud). Choose Typesense for: simpler SaaS applications where easy integration matters more than advanced analytics, small-to-medium datasets, or teams without ES expertise. Not ideal for petabyte-scale or advanced aggregation needs.

Q6. What is Meilisearch and what makes it different?

Meilisearch is a Rust-based, instant search engine focused on ease of use and sub-50ms search responses. Features: typo tolerance, faceting, filtering, geo search. Configuration is minimal — point at documents and search immediately. Designed for front-end search UIs. Choose for: product/document search in early-stage products, developer tools, documentation search. Limitations: less suitable for log analytics, complex aggregations, or high-write-throughput workloads compared to ES.

Q7. What is pgvector and when would you use it instead of a dedicated vector DB?

pgvector is a PostgreSQL extension that adds a `vector` column type and approximate nearest neighbor (ANN) indexes (IVFFlat, HNSW). Benefits: (1) No additional infrastructure — vectors live with your relational data. (2) Combine vector search with SQL filters (WHERE category='electronics'). (3) ACID guarantees with

the rest of your data. (4) Familiar PostgreSQL ecosystem. Choose pgvector when: data volume fits in PostgreSQL, you want to avoid a new service, and hybrid SQL+vector queries are important. Use dedicated vector DBs (Pinecone, Weaviate) for billion-vector scale.

Q8. Explain vector search and how it differs from keyword search.

Keyword search (BM25) matches documents based on term overlap — 'laptop' finds documents containing 'laptop'. Vector search matches by semantic similarity — 'portable computer' can match 'laptop' because their embeddings are close in vector space. Process: (1) Embed documents at index time using an ML model (OpenAI, BERT). (2) Store embedding as `dense_vector`. (3) At query time, embed the query, find k nearest vectors using ANN. Good for: synonym-heavy, conceptual, multilingual queries. Bad for: exact term matching, numeric ranges, structured filtering.

Q9. What is HNSW and why is it used for vector search?

HNSW (Hierarchical Navigable Small World) is a graph-based approximate nearest neighbor algorithm. It builds a hierarchical graph of vectors where each node connects to its nearest neighbors at multiple levels (like a skip list). At query time, it greedily traverses the graph to find approximate nearest neighbors. It's the state-of-the-art ANN algorithm: $O(\log N)$ query time with high recall. Used by: ES 8.x `dense_vector`, pgvector 0.5+, Weaviate, Qdrant. Trade-off: high memory (stores graph structure).

Q10. What is hybrid search and why is it better than either approach alone?

Hybrid search combines keyword (BM25) and vector (semantic) search results. BM25 excels at exact term matching, rare terms, product codes, proper nouns. Vector search excels at conceptual/semantic matching, synonyms, paraphrase. Together they cover each other's blind spots. Fusion methods: (1) RRF (Reciprocal Rank Fusion) — $\text{score} = \sum (1/(k + \text{rank}_i))$ per retriever. (2) Linear combination — $\alpha * \text{BM25_score} + (1 - \alpha) * \text{vector_score}$. ES 8.9+ supports RRF natively via `rank_rrf`. Most production search benefits from hybrid search over pure vector or keyword alone.

Q11. What are the main dedicated vector databases and their trade-offs?

Pinecone: managed-only SaaS, no self-hosting option, very easy, high cost at scale. Weaviate: open-source with cloud; graph-based; built-in ML modules; complex setup. Qdrant: open-source Rust-based; very fast; payload filtering; self-host or cloud. Chroma: open-source Python-first; great for prototyping; less mature production. All support HNSW and cosine/dot-product distance. Choose based on: team expertise, scale, whether you need self-hosting, and how tightly vector search is coupled to your other data.

Q12. How do you index and query embeddings in Elasticsearch?

Mapping: `dense_vector` with `dims` matching your model output (e.g., 1536 for OpenAI ada-002), `index: true`, `similarity: cosine` (or `dot_product`, `l2_norm`). Indexing: generate embedding at write time (call embedding API), store as `float[]`. Query: `knn` query with `query_vector`, `k` (results), `num_candidates` (ANN candidates). ES 8.x uses HNSW. Exact kNN: `script_score` with `cosineSimilarity` function (expensive, brute-force; only for small datasets or as rescore pass).

Q13. What is Retrieval-Augmented Generation (RAG) and how does ES fit in?

RAG is an AI pattern where a language model (LLM like GPT-4) is augmented with a retrieval step: (1) User asks a question. (2) ES retrieves relevant document chunks via hybrid search (BM25 + kNN). (3) Retrieved chunks are included in the LLM prompt as context. (4) LLM generates a grounded answer. ES is the 'retriever' in RAG. Its role: store document chunks with embeddings, provide fast and accurate retrieval, combine keyword and semantic search for high recall. Used in enterprise knowledge bases, customer support, code search.

Q14. How do you choose between Elasticsearch and a vector database for a RAG system?

For RAG use cases: If you have rich structured data alongside embeddings (product catalog, knowledge base with metadata, user-specific filtering), ES dense_vector + BM25 hybrid is ideal — one system for both. If you have pure vector search at very large scale (100M+ vectors), Pinecone/Qdrant may offer better performance and simpler setup. Many teams start with ES (since they already have it) and migrate to a dedicated vector DB only if ES can't meet the vector search SLA.

Q15. What is the difference between IVFFlat and HNSW in pgvector?

IVFFlat (Inverted File with Flat quantization): builds by clustering vectors into `lists` groups; at query time, searches the nearest `probes` clusters. Faster to build, less memory. Requires more data for good results (at least 1000 rows per list). HNSW: builds a multi-layer graph; higher build time and memory, but better recall and faster queries. Recommended: use HNSW for most workloads (available pgvector 0.5+). IVFFlat for budget-constrained or build-time-sensitive scenarios with large datasets.

Q16. What are the licensing implications of using Typesense vs Meilisearch?

Typesense: the server is GPL 3 (requires open-sourcing derivative works if distributed). For internal enterprise use or SaaS via Typesense Cloud, GPL 3 typically isn't an issue since you're not distributing the server. Meilisearch: MIT licensed — permissive, no restrictions. Elasticsearch: SSPL for the server; Apache 2.0 for client libraries (search-java, etc.). OpenSearch: Apache 2.0 for both server and clients — most permissive of the Lucene-based options.

Q17. How does Meilisearch handle relevance compared to Elasticsearch?

Meilisearch uses a rule-based ranking system rather than BM25: (1) typo count, (2) number of matching words, (3) proximity of matching words, (4) attribute weight (configurable per field), (5) exactness (exact match vs. prefix). This produces very intuitive results for e-commerce and documentation without complex scoring tuning. Elasticsearch's BM25 is more mathematically rigorous and tunable (k1, b, boosts) but requires more expertise to get right. For simple use cases, Meilisearch's defaults often produce good results out of the box.

Q18. When would you recommend keeping search in PostgreSQL (using FTS or pgvector)?

Use PostgreSQL full-text search (`tsvector`/`tsquery`) or pgvector when: (1) Data volume is small-medium (<10M rows, <1M vectors). (2) You want to avoid operational complexity of a separate search cluster. (3) Search needs to be transactionally consistent with the primary data. (4) You already use PostgreSQL managed services (RDS, Supabase, Neon). (5) Budget is constrained. Limitations: PostgreSQL FTS is less feature-rich than ES (no analyzers, no highlighting, limited scoring). pgvector ANN recall is lower than purpose-built vector DBs at scale.

Q19. What is the Elastic stack (ELK) and what are its components?

ELK stands for Elasticsearch + Logstash + Kibana. Elasticsearch: storage and search engine. Logstash: data pipeline (ingest, transform, enrich from any source). Kibana: visualization and management UI (dashboards, Discover, Dev Tools). Beats: lightweight data shippers (Filebeat for logs, Metricbeat for metrics, etc.). Modern stack is often called 'Elastic Stack'. Common in: log analytics (ELK), APM (Elastic APM), security (SIEM), and observability platforms. OpenSearch has equivalent: OpenSearch + Logstash + OpenSearch Dashboards (Kibana fork).

Q20. Describe a real-world scenario where you would recommend Typesense over Elasticsearch.

Scenario: a small-to-medium SaaS product needs a site-search feature for their documentation portal. The corpus is ~100k pages, updates are infrequent, the team is small, and there's no DevOps expertise for ES cluster management. Typesense recommendation: (1) Single-node or Typesense Cloud setup in minutes. (2) Typo tolerance works out of the box — 'authorizaton' matches 'authorization'. (3) Simple API key auth. (4) Predictable cost. ES would be over-engineering for this use case — Typesense's simpler operations let the

team focus on product rather than infrastructure.

Q21. What is the difference between dense vector and sparse vector search?

Dense vector: every dimension has a value; output of neural models (BERT, OpenAI); high-dimensional (768-1536 dims); captures semantic meaning. Used in ES ``dense_vector``, Pinecone, etc. Sparse vector: most dimensions are zero; represents term importance (like TF-IDF or BM25 weights per term); captures lexical relevance. SPLADE and SPLADEv2 are sparse learned models that generate sparse vectors matching learned expansion terms. Sparse vectors can be stored in ES as ``rank_features`` or ``sparse_vector`` fields. Hybrid: combine dense and sparse for best of both worlds.